

Instituto Tecnológico de Buenos Aires (ITBA)



25.27 - Sistemas Embebidos

Guías

Ignacio Sammartino

Índice

1 - Guía de Ejercicios N.º 1	3
1. 1 - Aprendiendo a leer la documentación del MCU MK64FN1M0VLL12	3
1. 2 - Verificando toolchain Kinetis	6
1. 3 - Editando el funcionamiento de Blink	6
1. 4 - Proyecto Pul2Switch	6
1. 5 - Interfaciando con la FRDM-K64F	6
1. 6 - Proyecto Baliza	7
2 - Guía de Ejercicios N.º 2	8
2. 1 - Aprendiendo a leer el Reference Manual del K64	8
2. 2 - Aprendiendo a leer el SDK	8
2. 3 - MyBlink	8
2. 4 - SysTick	8
2. 5 - Baliza_SysTick	8
2. 6 - Interrupciones de puerto	8
2. 7 - Baliza_IRQ	9

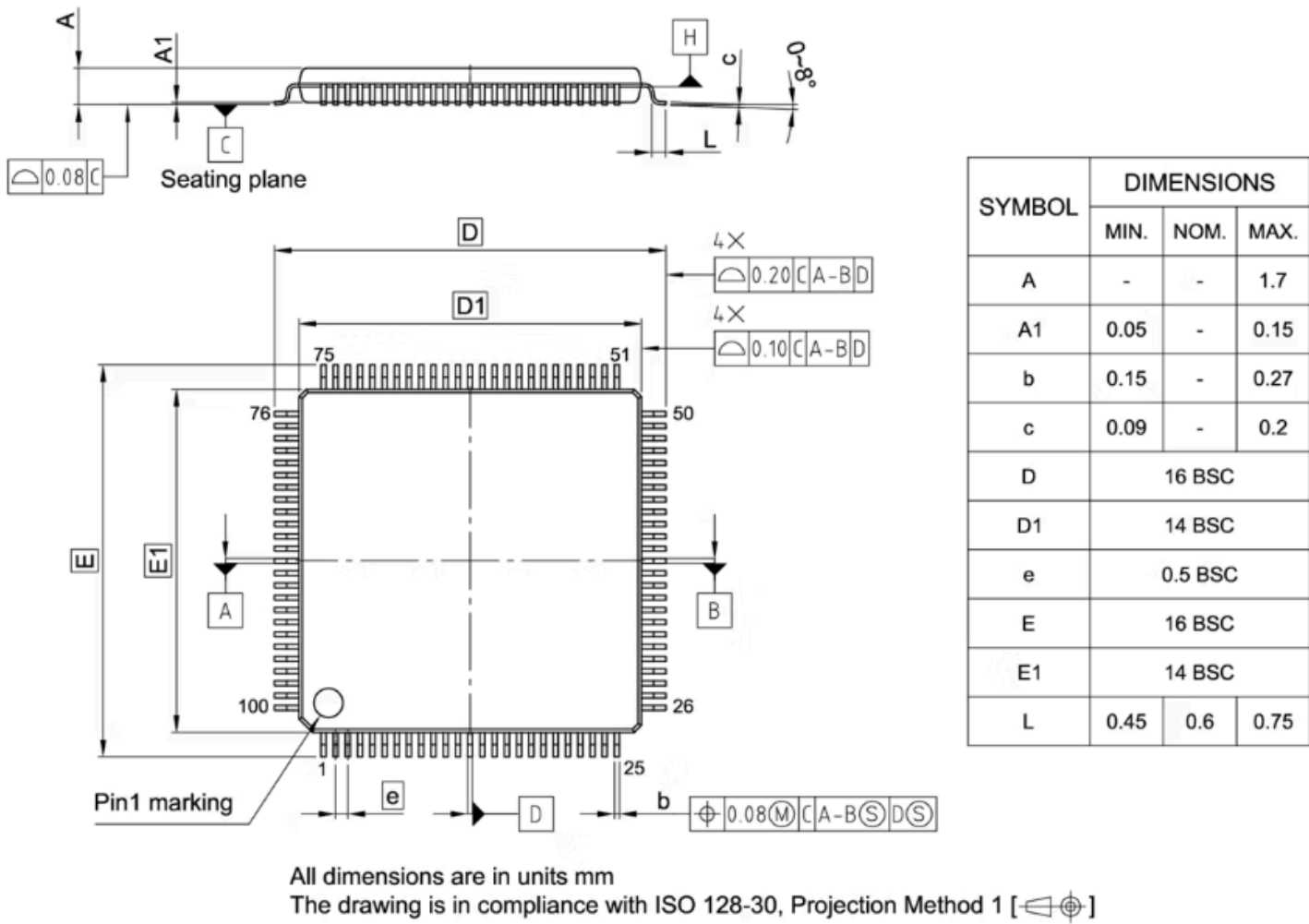
1 - Guía de Ejercicios N.º 1

Contestar las siguientes preguntas, indicando en cuál documento y sección se encuentra la respuesta:

1. 1 - Aprendiendo a leer la documentación del MCU MK64FN1M0VLL12

1. ¿En cuál número de pin del MCU se encuentra el puerto PTA12?

Dado que estamos utilizando la placa de evaluación FRDM-K64F, el encapsulado es el de 100 LQPF, por lo que segun la documentacion corresponde al pin 42.



144 LQFP	144 MAP BGA	121 XFB GA	100 LQFP	Pin Name	Default	ALT0	ALT1	ALT2	ALT3	ALT4	ALT5	ALT6	ALT7	EzPort
58	J7	—	—	PTA6	DISABLED		PTA6		FTM0_CH3		CLKOUT		TRACE_CLKOUT	
59	J8	—	—	PTA7	ADC0_SE10	ADC0_SE10	PTA7		FTM0_CH4				TRACE_D3	
60	K8	—	—	PTA8	ADC0_SE11	ADC0_SE11	PTA8		FTM1_CH0			FTM1_QD_PHA	TRACE_D2	
61	L8	—	—	PTA9	DISABLED		PTA9		FTM1_CH1	MII0_RXD3		FTM1_QD_PHB	TRACE_D1	
62	M9	J9	—	PTA10	DISABLED		PTA10		FTM2_CH0	MII0_RXD2		FTM2_QD_PHA	TRACE_D0	
63	L9	J4	—	PTA11	DISABLED		PTA11		FTM2_CH1	MII0_RXCLK	I2C2_SDA	FTM2_QD_PHB		
64	K9	K8	42	PTA12	CMP2_IN0	CMP2_IN0	PTA12	CAN0_TX	FTM1_CH0	RMII0_RXD1/ MII0_RXD1	I2C2_SCL	I2S0_TXD0	FTM1_QD_PHA	

Figura 3: Kinetis K64F Sub-Family Data-Sheet, PinOut

2. ¿Cuáles pines pueden funcionar como entradas analógicas?

1. Pines **analógicos dedicados** y de **alta resolución**:

- Pin 14: ADC0_DP1
- Pin 15: ADC0_DM1
- Pin 16: ADC1_DP1
- Pin 17: ADC1_DM1
- Pin 18: ADC0_DP0 / ADC1_DP3
- Pin 19: ADC0_DM0 / ADC1_DM3
- Pin 20: ADC1_DP0 / ADC0_DP3
- Pin 21: ADC1_DM0 / ADC0_DM3

2. Pines del **Puerto A**:

- PTA7 (Pin 59): ADC0_SE10
- PTA8 (Pin 60): ADC0_SE11
- PTA12 (Pin 42): CMP2_IN0
- PTA13 (Pin 43): CMP2_IN1
- PTA17 (Pin 47): ADC1_SE17

3. Pines del **Puerto B**:

- PTB0 (Pin 53): ADC0_SE8 y ADC1_SE8
- PTB1 (Pin 54): ADC0_SE9 y ADC1_SE9
- PTB2 (Pin 55): ADC0_SE12
- PTB3 (Pin 56): ADC0_SE13
- PTB10 (Pin 58): ADC1_SE14
- PTB11 (Pin 59): ADC1_SE15

4. Pines del **Puerto C**:

- PTC2 (Pin 72): ADC0_SE4b y CMP1_IN0

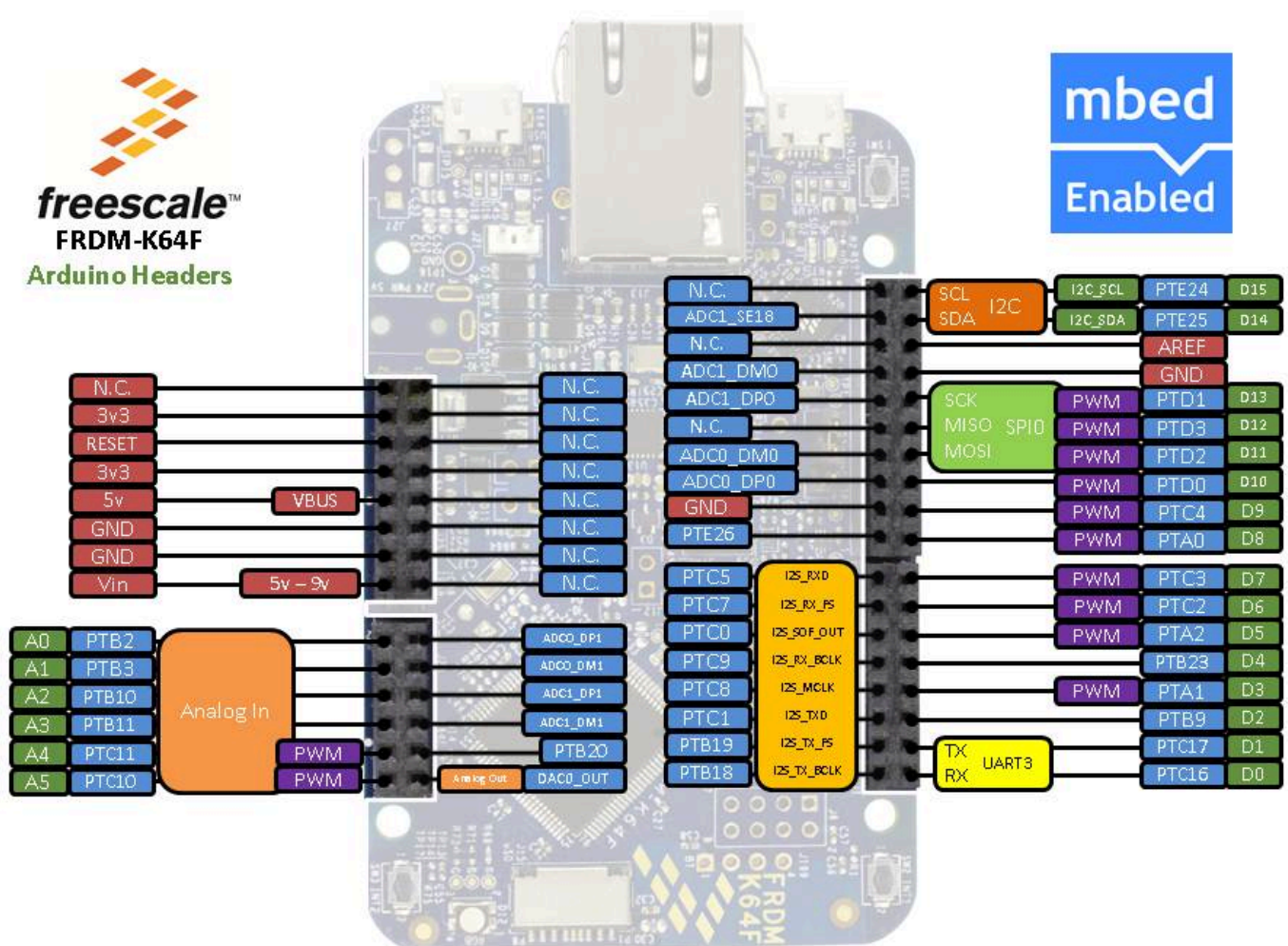
- PTC3 (Pin 73): CMP1_IN1
- PTC6 (Pin 78): CMP0_IN0

5. Pines del **Puerto E**:

- PTE1 (Pin 2): ADC1_SE5a
- PTE2 (Pin 3): ADC0_DP2 y ADC1_SE6a
- PTE3 (Pin 4): ADC0_DM2 y ADC1_SE7a
- PTE24 (Pin 31): ADC0_SE17
- PTE25 (Pin 32): ADC0_SE18

6. Otros pines con funciones **analógicas especiales**:

- Pin 26 (VREF_OUT): ADC1_SE18, CMP1_IN5 y CMP0_IN5
- Pin 27 (DAC0_OUT): ADC0_SE23 y CMP1_IN3



3. ¿Cuántos pines del puerto PTE se encuentran efectivamente disponibles en este modelo de MCU?

En el microcontrolador MK64FN1M0VLL12 con encapsulado 100 LQFP, se encuentran efectivamente disponibles 10 pines del puerto PTE.

4. ¿Cuál es el rango de valores de tensión para detectar un 0 y un 1 lógico en un pin I/O? ¿Se puede enviar 5 V a un pin?

Segun la hoja de datos en la seccion 3.6.1.1, dado que la tensión de alimentacion se debe encontrar entre 1.71V y 3.6V, segun la hoja de datos:

$$\begin{aligned}(V_H)_{\max} &= 3.6V \\ (V_H)_{\min} &= 1.13V \\ (V_L)_{\min} &= (V_L)_{\max} = V_{SSA} = \text{GND}\end{aligned}$$

5. ¿Cuánta es la máxima corriente que entrega un pin I/O?

1. 2 - Verificando toolchain Kinetis

1. Abrir e importar al MCUXpresso IDE el proyecto de ejemplo Blink.
2. Compilar y verificar que la compilación se complete correctamente.
3. Descargar el programa al MCU mediante el debugger; ejecutar y comprobar que parpadea el LED de la placa.
4. Colocar un breakpoint y ejecutar paso a paso. Visualizar la variable veces y el código en assembler del ciclo while; ejecutar instrucción por instrucción.
5. Modificar el nivel de optimización a “Optimize most”. ¿Cambia el comportamiento del programa? Investigar qué cambia y por qué.

1. 3 - Editando el funcionamiento de Blink

1. Modificar el programa para que titile el LED verde a 0.5 Hz (1 segundo encendido, 1 segundo apagado).
2. Obtener una captura de osciloscopio del pin del LED.

1. 4 - Proyecto Pul2Switch

1. Basado en Blink, crear un nuevo proyecto llamado Pul2Switch.
2. Modificar el programa para que el LED cambie de estado en cada pulsación del pulsador SW3 (es decir, por flanco).
3. ¿El LED cambia de estado siempre que se presiona el pulsador? Si no, investigar por qué (rebote, configuración de pull-ups, lógica inversa, etc.).
4. Modificar el programa para usar el pulsador SW2 deshabilitando el pull-up por software. ¿Sigue funcionando? Investigar por qué.

1. 5 - Interfaciando con la FRDM-K64F

1. Conectar la FRDM-K64F a un protoboard para usar un pulsador externo y un LED amarillo externo. Colocar el pulsador en PTC9 y el LED en PTB23.

2. Usar una resistencia de $330\ \Omega$ en serie con el LED y una resistencia de $330\ \Omega$ en serie con el pulsador para evitar cortocircuitos.
3. Modificar el programa para el nuevo conexionado y verificar depurando paso a paso.
4. Luego mover el pulsador a PTC0 y el LED a PTA0; adaptar el programa y verificar.

1. 6 - Proyecto Baliza

1. Crear un proyecto nuevo llamado Baliza que simule la baliza de un automóvil.
2. Al pulsar SW3, el LED amarillo externo debe parpadear a 0.5 Hz. Al volver a pulsar, debe apagarse. El LED rojo de la placa debe indicar cuando la baliza está activada.
3. Asegurarse de que el programa no pierda eventos de pulsado del pulsador (manejo de rebote y detección por flancos).

2 - Guía de Ejercicios N.º 2

Contestar las siguientes preguntas, indicando en cuál documento y sección se encuentra la respuesta:

2.1 - Aprendiendo a leer el Reference Manual del K64

1. ¿Cuál es la dirección absoluta donde se encuentra el registro PCR del pin PTA12?
2. ¿Cuál bit del registro PCR corresponde al **Interrupt Status Flag**?
3. Luego del **reset** los pines del puerto B: ¿cómo tienen configurada su dirección (entrada o salida)? ¿Y cómo tienen configurado el **slew-rate** (activado o no)? ¿Y cómo tienen configurado el **pull** (desactivado, activado **pullup** o activado **pulldown**)?

2.2 - Aprendiendo a leer el SDK

1. Analizar la estructura Port_Type. ¿En cuál archivo se encuentra declarado? ¿Por qué tiene campos reservados?

2.3 - MyBlink

1. Basado en Blink, crear un proyecto nuevo llamado MyBlink.
2. Quitar el archivo gpio.o y escribir su propio archivo gpio.c, de tal forma que se implementen las funciones (servicios) de gpio.h.
3. Verificar el correcto funcionamiento de gpio.c en los proyectos Pul2Switch y Baliza.

2.4 - SysTick

1. Escribir su propio archivo SysTick.c, de tal forma que se implementen las funciones (servicios) de SysTick.h.
2. Modificar el proyecto MyBlink para que el retardo se implemente con el SysTick en lugar de un ciclo. Analizar cómo hacer para que el retardo no sea bloqueante. Verificar el correcto funcionamiento con un osciloscopio.
3. Agregar un pin de testeo (**tespoint** o TP) que se encienda mientras se ejecuta la interrupción. Medir el tiempo que dura la ISR y cuánto representa porcentualmente.

2.5 - Baliza_SysTick

1. Basado en los proyectos anteriores, crear el proyecto Baliza_SysTick que implemente el programa de la baliza utilizando SysTick como base de tiempo. Utilizar el pulsador SW3 y realizar un muestreo periódico de su estado para detectar eventos.

2.6 - Interrupciones de puerto

1. Escribir un nuevo archivo gpio.c, de tal forma que se implementen las funciones (servicios) de manejo de interrupciones del nuevo gpio.h.
2. Escribir un programa de prueba (**testbench**) que verifique el correcto funcionamiento de la nueva implementación de gpio.
3. Agregar un TP que se encienda mientras se ejecuta la interrupción. Medir el tiempo que dura la ISR.

2. 7 - Baliza_IRQ

1. Basado en los proyectos anteriores, crear el proyecto Baliza_IRQ que implemente el programa de la baliza utilizando el nuevo gpio para leer el pulsador. Utilizar el pulsador SW2 y detectar evento de presionado por interrupción.